

Algo PBX — Infrastructure Guide 1 of 2

Deploying on a Linux VM (VirtualBox)

From a blank Windows PC running Oracle VirtualBox to a running Algo PBX stack: creating the VM, understanding and clearing port conflicts, Docker Compose, secrets, TLS certificates, the first admin account at `/setup`, firewall, and backups.

Written for a non-technical operator. Every command in this guide is meant to be copy-pasted exactly as printed — nothing here requires guessing.

Next step: "Configuring Credentials, Dinstar & Going Live" (Guide 2 of 2) wires up `/admin/settings`, connects the physical Dinstar gateway, and cuts your domain over.

This guide mirrors `DEPLOYMENT.md` in the repository — if they ever disagree, the repository file wins.

Contents

1. What you are building, and what this machine needs
2. Ports — the chapter that prevents most failures
3. Creating the `algo_pbx` VM in VirtualBox
4. First boot — finding and fixing the VM's network address
5. Install Docker Engine + Compose
6. Clone and generate secrets
7. Router port forwarding
8. TLS certificates
9. Build and start
10. First run — `/setup`
11. Firewall
12. Ongoing operations
13. Appendix A — port cheat sheet

1. What you are building, and what this machine needs

Algo PBX is seven Docker containers running together on one Linux VM: **postgres** (database), **coturn** (helps agents' browsers connect through home/office routers), **asterisk** (the phone system engine), **web** (the softphone + admin app you use in a browser), **cdr-listener** (call-log writer), **openwa** (WhatsApp), and **caddy** (the front door — it terminates HTTPS). When this guide is done, agents anywhere sign into `https://your-domain` and place/receive calls through a physical Dinstar GSM gateway (wired up in Guide 2).

Before you start, confirm on the Windows host machine:

- Oracle VirtualBox is installed, and virtualization is enabled in the PC's BIOS/UEFI (VirtualBox will refuse to start a VM otherwise — it shows a clear error naming "VT-x/AMD-V" if this is off).
- At least **4 free CPU cores, 8 GB free RAM, and 80 GB free disk** for the new VM — on top of whatever your existing `ubuntuserver` VM already uses.
- An Ubuntu Server 24.04 LTS ISO downloaded (from `ubuntu.com/download/server`).

Your machine already runs another VM. The screenshot below is this exact setup: an existing `ubuntuserver` VM (running, with Coolify) already forwards host ports **8000, 80, and 443**, and a second VM named `algo_pbx` is where Algo PBX will actually run. Chapter 2 explains why this matters and exactly what to do about it — read it before creating the VM in Chapter 3.

2. Ports — the chapter that prevents most failures

2.1 — What a "port" is, in three sentences

A port is a numbered door a program listens on for network traffic — port 443 is the standard door for HTTPS websites, for example. Only one program on a given machine can hold a given port open at a time; if a second program tries to use the same port, it fails to start (or, with VirtualBox NAT forwarding, VirtualBox refuses to add the rule). A port can be occupied in **three different places** in this setup, and each is checked differently: the **Windows host** itself, the **Ubuntu VM** running inside it, and your **office/home router** (which decides what reaches the VM from the internet).

2.2 — The full port matrix

These are every port the Algo PBX stack uses. "Public?" means it must be reachable from the internet, not just your LAN.

Port	Protocol	Purpose	Public?	What breaks if it's blocked
80	tcp	Certbot challenges, redirect to HTTPS	yes	Free TLS certificate issuance/renewal fails
443	tcp	Web app UI (Caddy → web:3000)	yes	The website / softphone doesn't load at all
8089	tcp	Asterisk WSS signaling for agent softphones	yes	Agents' browser softphones never register — no dial tone
3478	tcp+udp	Coturn STUN/TURN listener	yes	Agents behind strict NAT/firewalls can't reach media
5349	tcp	Coturn TURN over TLS	yes	TURN relay fails on networks that block plain TURN
10000–20000	udp	Asterisk RTP media (the actual voice audio)	yes	Calls connect but are completely silent
20001–30000	udp	Coturn relay range	yes	Relayed calls (agents behind strict NAT) are silent
5060	udp	SIP to the Dinstar trunk — Tailscale ONLY	no	N/A — this must never be internet-facing (see §4.4 hard constraint)
5038	tcp	Asterisk AMI — Docker bridge ACL only	never	N/A — internal control channel, never expose it
5432	tcp	PostgreSQL — loopback publish only	never	N/A — database, never expose it

2.3 — Checking what's already using a port on the Windows host

Run these in a Windows PowerShell window (right-click Start → "Terminal (Admin)").

```
# Is anything already listening on port 443?
netstat -ano | findstr ":443"

# Same, with more detail (address, state, and the owning process ID):
Get-NetTCPConnection -LocalPort 443 | Select-Object LocalAddress,State,OwningProcess
```

```
# Turn that process ID into a program name:
Get-Process -Id <PID>

# Windows silently reserves some ports for Hyper-V/WSL - a port can look "free"
# and still refuse to bind. Check for hidden exclusions:
netsh interface ipv4 show excludedportrange protocol=tcp
```

Check your other VM's forwarding rules directly. This is exactly the situation in the screenshot on the cover of this guide — run this from PowerShell to list every port `ubuntuuserver` already forwards, so you know for certain which host ports are taken before you touch `algo_pbx` :

```
VBoxManage showvminfo "ubuntuuserver" --machinereadable | findstr Forwarding
```

In the reference setup this prints three rules claiming host ports **8000, 80, and 443** — all mapped to `ubuntuuserver`, not `algo_pbx`.

2.4 — Checking what's already using a port inside the Ubuntu VM

```
# List every listening port with the program name attached:
sudo ss -tulpn

# Narrow it to exactly the ports this stack needs:
sudo ss -tulpn | grep -E ':(80|443|8089|3478|5349|5060|5038|5432)\b'

# Alternative, if ss isn't installed:
sudo lsof -i :443

# List ports already published by any running Docker containers:
docker ps --format '{{.Names}} {{.Ports}}'
```

Common squatters and how to clear them:

Program	Port it takes	Fix
Apache2 / Nginx (pre-installed on some Ubuntu images)	80, 443	<code>sudo systemctl disable --now apache2</code> or <code>nginx</code>
<code>systemd-resolved</code>	53 (not used here, but a classic surprise)	Not relevant to this stack — ignore unless troubleshooting DNS
A leftover Coolify/Traefik install	80, 443, 8000	<code>docker stop \$(docker ps -q)</code> then remove that stack, or dedicate a separate VM (recommended — see Chapter 3)

2.5 — Choosing a free port, if you need to change one

1. Pick any number in the 1024–65535 range that isn't already listed by the checks in §2.3/§2.4.
2. Avoid anything inside a Windows-excluded range shown by `netsh interface ipv4 show excludedportrange protocol=tcp`.
3. Verify it's actually free with the same command from §2.4 before committing to it.

Default	Suggested alternate
443	8443

80	8081
8089	8090
3478	3478 (leave as-is — STUN/TURN clients expect the standard port)

2.6 — What is movable, and what is not

Movable (only the Caddy web-facing ports, and only if you have a real reason to): changing this means editing **all** of the following consistently — `docker-compose.yml` 's `caddy.ports` section, the site address in `Caddyfile`, `AUTH_URL` / `NEXT_PUBLIC_SIP_WS_SERVER` / `NEXT_PUBLIC_TURN_SERVER` in `.env`, the matching line in `scripts/setup-firewall.sh`, and your router's forwarding rule. A non-443 port then has to appear in every URL agents type.

Not movable — leave these exactly as documented: the RTP range (`10000–20000/udp`) and Coturn relay range (`20001–30000/udp`) because Asterisk and Coturn run in `network_mode: host`; `5060/udp` because it is reachable only over the Tailscale mesh by design (a hard constraint — never put it on a public port, see `LLM.md §2`); `5038` (AMI) and `5432` (Postgres) because they are never meant to be reachable from outside the Docker host at all.

2.7 — Symptom → cause → fix

Symptom	Likely cause	Fix
<code>bind: address already in use</code> when running <code>docker compose up</code>	Another program (or another container) already holds that port inside the VM	Run §2.4's checks, stop the conflicting service or change the port per §2.5/§2.6
VirtualBox refuses to add a forwarding rule / silently doesn't work	Another VM (like <code>ubuntuserver</code>) already forwards that exact host port	Run §2.3's <code>VBoxManage</code> check, pick a different host port, or (recommended) use Bridged networking so no forwarding is needed — Chapter 3
Browser shows nothing / times out on <code>https://your-domain</code>	Port 443 not actually reachable — VM firewall, router, or DNS issue	Confirm <code>sudo ss -tulpn</code> shows Caddy on 443 inside the VM, then work outward: VM firewall (Ch.11) → router forwarding (Ch.7) → DNS record
Call connects but there's total silence	The RTP UDP range (10000–20000) or Coturn relay range isn't actually reaching the VM	Re-check router forwarding for those UDP ranges (Ch.7) and that nothing rewrites/blocks UDP in between

3. Creating the `algo_pbx` VM in VirtualBox

1. VirtualBox Manager → **New**. Name it `algo_pbx`, type **Linux**, version **Ubuntu (64-bit)**.
2. Memory: **8192 MB**. Processors: **4**.
3. Create a new virtual hard disk, VDI, dynamically allocated, **80 GB**.
4. Before starting it: select `algo_pbx` → **Settings** → **Network** → **Adapter 1**.

Set "Attached to:" = Bridged Adapter, bound to your physical network card (Wi-Fi or Ethernet) — **not** NAT. This is the recommended and supported setup for this guide.

Why not NAT with port forwarding, like `ubuntuserver` uses? NAT forwarding needs one explicit rule per port. The RTP range alone is 10,000 ports wide, plus another 10,000 for the Coturn relay range — nobody hand-writes 20,000 forwarding rules, and VirtualBox's NAT engine isn't built for it. In practice a NAT-forwarded Algo PBX will let a browser load the UI but every call will connect with no audio. Bridged Adapter instead puts `algo_pbx` directly on your LAN with its own IP address, exactly like a physical machine plugged into your router — no forwarding rules between the host and the VM at all, and it also means `algo_pbx` can never collide with whatever `ubuntuserver` already forwards (8000/80/443 in the screenshot on the cover).

If `algo_pbx` already has NAT port-forwarding rules from a template or clone, open **Settings** → **Network** → **Adapter 1** → **Advanced** → **Port Forwarding** and delete all of them once Bridged Adapter is selected — they do nothing under Bridged mode but are worth removing to avoid confusion later.

- Attach the Ubuntu Server 24.04 LTS ISO under **Settings** → **Storage**, then **Start** the VM and run through the standard Ubuntu Server installer (defaults are fine; enable OpenSSH server when offered).

4. First boot — finding and fixing the VM's network address

```
# Inside the VM, once installed and logged in:
hostname -I
# or:
ip -4 addr show
```

Note the IP address shown (e.g. `192.168.1.50`) — that's the VM's address on your LAN, reachable from any device on the same network, including your Windows host.

Reserve this IP as a static DHCP lease in your router's admin page (look for "DHCP Reservation" or "Static Lease" against the VM's MAC address) so it never changes after a reboot — every later step (router forwarding, DNS, certificates) depends on this IP staying fixed.

```
# From the Windows host, confirm you can reach it:
ssh youruser@192.168.1.50
```

5. Install Docker Engine + Compose

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker

# Prove it works:
docker run hello-world
```


6. Clone and generate secrets

```
git clone <your-fork-url> algo-pbx
cd algo-pbx
cp .env.example .env

# Generate the bootstrap secrets:
openssl rand -hex 32      # -> SETTINGS_ENCRYPTION_KEY
openssl rand -base64 33   # -> AUTH_SECRET
openssl rand -hex 32      # -> COTURN_AUTH_SECRET
openssl rand -hex 32      # -> AMI_SECRET, CDR_AMI_SECRET, CDR_INGEST_SECRET
```

Fill in the **bootstrap** section at the top of `.env`: Postgres credentials, `SETTINGS_ENCRYPTION_KEY`, `AUTH_SECRET` / `AUTH_URL`, AMI credentials, `VM_PUBLIC_DOMAIN` / `VM_PUBLIC_IP` / `VM_PRIVATE_IP` (use the LAN IP from Chapter 4 for `VM_PRIVATE_IP`), `COTURN_AUTH_SECRET`, `DINSTAR_LAN_IP`.

Do **not** fill in the RUNTIME SETTINGS section (Resend, OpenWA, Meta, Dinstar SMS, Firebase, CRM) unless you want a fully non-interactive deploy — all of it is configurable later in the browser at `/admin/settings` and takes effect immediately there.

7. Router port forwarding

Log into your office/home router's admin page and forward the **Public? = yes** rows from the §2.2 matrix to the VM's LAN IP (the static reservation from Chapter 4):

Port	Protocol	Forward to
80	tcp	the VM's LAN IP, e.g. <code>192.168.1.50</code>
443	tcp	
8089	tcp	
3478	tcp+udp	
5349	tcp	
10000–20000	udp	
20001–30000	udp	

Never forward these: 5060 (Dinstar SIP — Tailscale only), 5038 (AMI), 5432 (Postgres). Forwarding any of these to the public internet is a security hole regardless of how the port matrix reads elsewhere.

Your ISP connection must have a real public IP, not CGNAT (Carrier-Grade NAT) — if your router's WAN IP differs from what `curl ifconfig.me` shows from inside the VM, you're behind CGNAT and port forwarding will not work; contact your ISP for a public IP or use a reverse-tunnel provider instead.

8. TLS certificates

One certificate pair serves everything — Caddy (443), Asterisk WSS (8089), Coturn TLS (5349). Place them at `pbx_configs/keys/fullchain.pem` + `privkey.pem`. Self-signed is fine for a first smoke test but agents' browsers will reject self-signed `wss://`.

```
# Recommended once DNS resolves: DNS-01 via Cloudflare API token
sudo apt install -y certbot python3-certbot-dns-cloudflare
# put "dns_cloudflare_api_token = ..." in /etc/letsencrypt/cloudflare.ini (chmod 600)
sudo certbot certonly --dns-cloudflare \
  --dns-cloudflare-credentials /etc/letsencrypt/cloudflare.ini -d your-domain.example.com
sudo cp /etc/letsencrypt/live/your-domain.example.com/fullchain.pem pbx_configs/keys/
sudo cp /etc/letsencrypt/live/your-domain.example.com/privkey.pem pbx_configs/keys/
```

9. Build and start

```
bash vendor/openwa/prepare.sh    # fetches the pinned OpenWA upstream commit

docker compose build
docker compose up -d
docker compose logs -f web        # watch for a clean startup
```

- The stack runs seven containers: postgres, coturn, asterisk, web, cdr-listener, openwa, caddy.
- If Postgres's data volume predates this deploy, create OpenWA's database by hand once: `docker compose exec postgres psql -U $POSTGRES_USER -d postgres -f /docker-entrypoint-initdb.d/01-create-openwa-db.sql`
- Migrations run automatically when the web container starts.

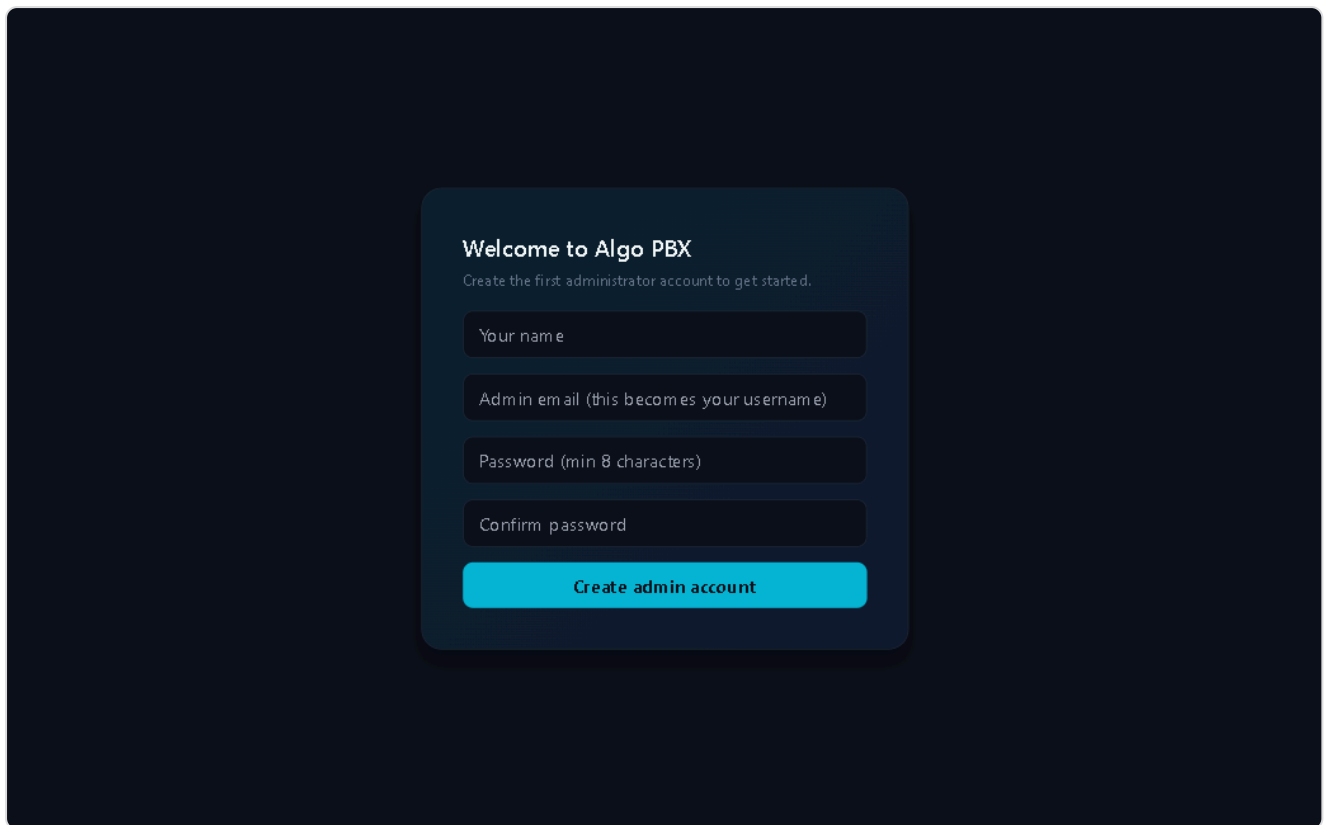
Verify every port actually bound before moving on:

```
sudo ss -tulpn
```

You should see something listening on 80, 443, 8089, 3478, and 5349, plus Asterisk holding the RTP range. If any are missing, go back to Chapter 2 — do not proceed to `/setup` with a half-bound stack.

10. First run — `/setup`

Browse to `https://your-domain/setup`. The page is reachable only until the first ADMIN account exists. It refuses to proceed without `SETTINGS_ENCRYPTION_KEY`, naming the exact fix command. After setup you land on `/admin/settings` — configure each service with its **Test connection** button (full detail in Guide 2).



Live screenshot: /setup rendering the first-administrator creation form.

11. Firewall

```
sudo bash scripts/setup-firewall.sh # applies the port matrix above via ufw
```

Check the SSH port inside the script first. 5060 is deliberately NOT opened publicly — the Dinstar trunk is reached over the Tailscale interface only. This firewall protects the VM itself; the router forwarding from Chapter 7 controls what reaches the VM from the internet in the first place — both layers matter. Docker publishes bypass ufw's normal rule chain, so the script also installs explicit `DOCKER-USER` rejects for 5432 and 5038 to make that guarantee explicit.

12. Ongoing operations

Task	How
Application update	<code>git pull && docker compose build && docker compose up -d</code> (migrations run automatically)
Credential rotation	<code>/admin/settings</code> in the browser — immediate, no restart
Telephony config change	Edit <code>.env</code> / <code>pbx_configs/</code> , then restart the affected container
Backups	<code>scripts/backup.sh</code> on cron — dumps both databases plus recordings, voicemail, photos, WhatsApp sessions. Do a restore drill before go-live.
Readiness check	<code>/admin/system</code> — live green/red checks for every dependency

Appendix A — Port cheat sheet

Every check-command from Chapter 2 in one place — copy-paste this block whenever something isn't reachable.

```
### On the Windows host (PowerShell, as Admin) ###
netstat -ano | findstr ":443"
Get-NetTCPConnection -LocalPort 443 | Select-Object LocalAddress,State,OwningProcess
Get-Process -Id <PID>
netsh interface ipv4 show excludedportrange protocol=tcp
VBoxManage showvminfo "ubuntuserver" --machinereadable | findstr Forwarding

### Inside the algo_pbx Ubuntu VM ###
sudo ss -tulpn
sudo ss -tulpn | grep -E ':(80|443|8089|3478|5349|5060|5038|5432)\b'
sudo lsof -i :443
docker ps --format '{{.Names}} {{.Ports}}'
```

Where next

Open **Guide 2 of 2 — Configuring Credentials, Dinstar & Going Live** to wire up `/admin/settings`, connect the physical Dinstar gateway, and cut your domain over to Cloudflare with free SSL. Its Appendix B covers the telephony-side ports (Dinstar over Tailscale) that build on this chapter's port matrix.